

Batocera v42 Volume Button

Setup & Troubleshooting Reference

Raspberry Pi 5 GPIO Pin (Physical) 14, 16, 18

GPIO wiring - 3 wires only

Button Pin	Pi 5 Physical Pin	Function
Button 1 Leg 1 + Button 2 Leg 1 (Daisy Chain)	Pin 14	GND
Button 1 Leg 2	Pin 16	Volume DOWN
Button 2 Leg 2	Pin 18	Volume UP

1. Batocera Configuration

1.1 Check GPIO library availability:

Check this library on Batocera Alt + F5 or via SSH:

```
python3 -c "import lgpio; print('lgpio OK')" 2>/dev/null || echo "lgpio not available"
python3 -c "import RPi.GPIO; print('RPi.GPIO OK')" 2>/dev/null || echo "RPi.GPIO not available"
```

Note which one prints OK - the script handles both automatically but good to know in advance.

1.2 Create the script

```
mkdir -p /userdata/system/scripts
nano /userdata/system/scripts/volume_buttons.py
```

Paste the complete final script:

```
import subprocess
import time
```

```

import threading

try:
    import lgpio
    USE_LGPIO = True
except ImportError:
    import RPi.GPIO as GPIO
    USE_LGPIO = False

# — Configuration —————
VOL_UP_PIN      = 24
VOL_DOWN_PIN    = 23
SHORT_STEP      = 5
LONG_STEP       = 2
LONG_PRESS_TIME = 0.5
RAMP_INTERVAL   = 0.1
MUTE_WINDOW     = 0.05
DEBOUNCE_MS     = 500      # increased debounce
BOUNCE_TIME     = 300
# —————

# Thread safety — only one button action at a time
_lock = threading.Lock()
_last_trigger_time = {VOL_UP_PIN: 0, VOL_DOWN_PIN: 0}
_active_threads = 0
MAX_THREADS = 2  # never allow more than 2 concurrent button threads

if USE_LGPIO:
    h = lgpio.gpiochip_open(0)
    lgpio.gpio_claim_input(h, VOL_UP_PIN, lgpio.SET_PULL_UP)
    lgpio.gpio_claim_input(h, VOL_DOWN_PIN, lgpio.SET_PULL_UP)
else:
    GPIO.setmode(GPIO.BCM)
    GPIO.setup(VOL_UP_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)
    GPIO.setup(VOL_DOWN_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)

def read_pin(pin):
    if USE_LGPIO:
        return lgpio.gpio_read(h, pin)
    else:
        return GPIO.input(pin)

def set_volume(delta):
    sign = '+' if delta > 0 else '-'
    subprocess.run(
        ['batocera-audio', 'setSystemVolume', '{}{}'.format(sign,
abs(delta))],
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL
    )

```

```

    )

def get_volume():
    result = subprocess.run(
        ['batocera-audio', 'getSystemVolume'],
        capture_output=True,
        text=True
    )
    try:
        return int(result.stdout.strip())
    except (ValueError, AttributeError):
        return 50

def toggle_mute():
    current = get_volume()
    target = '0' if current > 0 else '50'
    subprocess.run(
        ['batocera-audio', 'setSystemVolume', target],
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL
    )

def handle_button(pin, direction):
    global _active_threads
    try:
        # Verify pin is actually still LOW – reject ghost triggers
        if read_pin(pin) != 0:
            return

        # Check for simultaneous press (mute)
        time.sleep(MUTE_WINDOW)
        if read_pin(VOL_UP_PIN) == 0 and read_pin(VOL_DOWN_PIN) == 0:
            toggle_mute()
            while read_pin(VOL_UP_PIN) == 0 or read_pin(VOL_DOWN_PIN)
== 0:
                time.sleep(0.05)
            return

        press_time = time.time()
        long_press_triggered = False

        while read_pin(pin) == 0:
            held_duration = time.time() - press_time
            if held_duration >= LONG_PRESS_TIME:
                long_press_triggered = True
                set_volume(direction * LONG_STEP)
                time.sleep(RAMP_INTERVAL)

        if not long_press_triggered:

```

```

        set_volume(direction * SHORT_STEP)

    finally:
        with _lock:
            _active_threads -= 1

def button_callback(chip, gpio, level, tick):
    global _active_threads

    now = time.time()

    # Software debounce - reject triggers within DEBOUNCE_MS of last
    trigger
    with _lock:
        last = _last_trigger_time.get(gpio, 0)
        if (now - last) < (DEBOUNCE_MS / 1000.0):
            return
        _last_trigger_time[gpio] = now

    # Thread limit - reject if too many active threads
    if _active_threads >= MAX_THREADS:
        return
    _active_threads += 1

    direction = 1 if gpio == VOL_UP_PIN else -1
    t = threading.Thread(
        target=handle_button,
        args=(gpio, direction),
        daemon=True
    )
    t.start()

# Register callbacks
if USE_LGPIO:
    lgpio.gpio_claim_alert(h, VOL_UP_PIN, lgpio.FALLING_EDGE)
    lgpio.gpio_claim_alert(h, VOL_DOWN_PIN, lgpio.FALLING_EDGE)
    cb1 = lgpio.callback(h, VOL_UP_PIN, lgpio.FALLING_EDGE,
button_callback)
    cb2 = lgpio.callback(h, VOL_DOWN_PIN, lgpio.FALLING_EDGE,
button_callback)
else:
    GPIO.add_event_detect(
        VOL_UP_PIN, GPIO.FALLING,
        callback=button_callback,
        bouncetime=BOUNCE_TIME
    )
    GPIO.add_event_detect(
        VOL_DOWN_PIN, GPIO.FALLING,
        callback=button_callback,

```

```

        bouncetime=BOUNCE_TIME
    )

try:
    while True:
        time.sleep(0.1)
except KeyboardInterrupt:
    pass
finally:
    if USE_LGPIIO:
        cb1.cancel()
        cb2.cancel()
        lgpio.gpiochip_close(h)
    else:
        GPIO.cleanup()

```

Save and exit: Ctrl+X → Y → Enter

1.3 Use `custom.sh` for script auto start:

Use `custom.sh` which is Batocera's official autostart hook:

```
| nano /userdata/system/custom.sh
```

Add:

```
| #!/bin/bash
| python3 /userdata/system/scripts/volume_buttons.py &

```

Save with Ctrl+X → Y → Enter

Then:

```
| chmod +x /userdata/system/custom.sh

```

Test manually before rebooting. Run the script directly first:

```
| python3 /userdata/system/scripts/volume_buttons.py &

```

Now test all three behaviours:

- Short tap Vol Up button → volume increases 5%
- Short tap Vol Down button → volume decreases 5%
- Hold Vol Up → volume ramps up continuously
- Hold Vol Down → volume ramps down continuously
- Press both simultaneously → mute toggle

If everything works, kill the manual instance:

```
| pkill -f volume_buttons.py

```

Reboot:

```
| reboot

```

After reboot SSH back in and check:

```
pgrep -f volume_buttons.py && echo "Auto-started OK" || echo "Did not auto-start"
```

2. Problems Faced

- A lot of ghosting was occurring due to AMP interference, so I had to increase Debounce so that constant interference doesn't cause volume level changes. Will also need to add 100nF Ceramic Capacitors to further increase filtering.
- Earlier volume script `S99volume.sh` caused buffers to get stored in RAM causing RAM to run out of memory and crashing Batocera, so switched to the latest script.